

Recommendation Systems for Personalized Content Discovery

A personalized movie recommendation engine built on the Netflix Prize dataset — learning user taste from ~100M ratings to predict unseen ratings and generate explainable Top-10 lists.

Dataset

Netflix Prize (100.5M ratings)

Core Models

Baseline · Item-KNN · SVD · Hybrid

Best RMSE

0.8505 (SVD)

Best MAP@10

0.0610 (Hybrid)

Prepared by : **Anugra Chahande**
Enrollment No. :**24118006**

· June 2026 · Source & reproduction steps in the accompanying GitHub repository

Abstract — This report develops and compares four rating-prediction recommenders on the Netflix Prize dataset. After a full exploratory analysis establishing the dataset's positive rating skew, long-tail activity, temporal drift and **~99% sparsity**, we train a bias-only **baseline**, an **item-based KNN**, and a **matrix-factorization SVD** using a leakage-free per-user time split. SVD achieves the best rating accuracy (**RMSE 0.8505**), and a lightweight **title-based hybrid re-ranker** lifts ranking quality by **+57.6%** over plain SVD (**MAP@10 0.0387 → 0.0610**). We add an explanation layer, a cold-start strategy, and an interactive dashboard, and close with honest success/failure analysis.

100.5M

ratings · 480K users · 17.8K movies

1.18%

matrix density (highly sparse)

0.8505

best RMSE (SVD)

0.0610

best MAP@10 (Hybrid)

1 Problem Understanding

A streaming catalogue holds ~17,770 titles — far more than any user can browse. The platform must therefore **learn each person's taste** from their rating history and surface the few titles they are most likely to enjoy. Using the Netflix Prize data (~100M ratings, 480,189 users, 17,770 movies, 1–5 stars), the system must (1) learn user preferences, (2) predict the rating a user would give an unseen movie — scored by **RMSE**, (3) produce a personalized **Top-10 list** — scored by **MAP@10** with a movie counted relevant when the true rating is ≥ 3.5 , and (4) **explain** its recommendations.

The central obstacle is **sparsity**: the full user×movie table has ≈ 8.53 billion cells but only ~100M are filled, so it can neither be stored densely nor trained on in full on commodity hardware. We therefore work on a faithful, sparsity-preserving subset (§3) and choose models that exploit — rather than fight — the missing data.

2 Exploratory Data Analysis

We examine the four properties the problem statement calls out — the rating distribution, user activity, movie popularity, and sparsity — and translate each finding into a concrete modelling decision.

2.1 Rating distribution & temporal drift

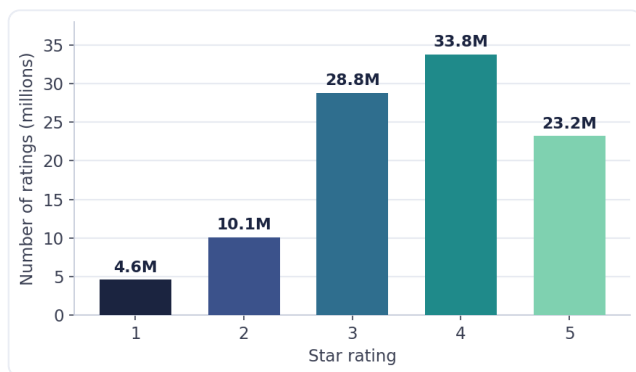


Fig 1 — Ratings skew positive: 4★ is the mode; global mean ≈ 3.604 .

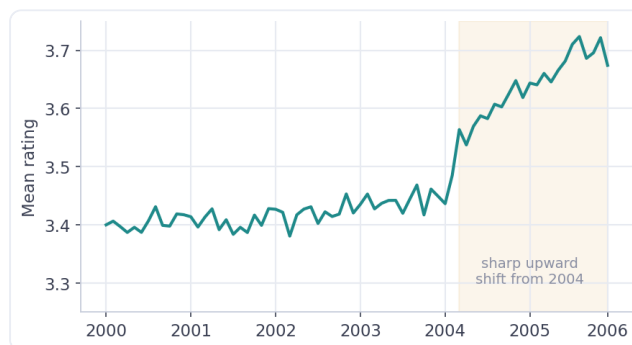


Fig 2 — Mean rating drifts upward over time, with a sharp shift from 2004.

Ratings lean strongly positive — **56.6%** are 4★ or 5★ and the global mean is **3.604** — because users mostly rate films they chose to watch. The series mean also **drifts upward over the years**.

Modelling implication

Predicting the global average is already a fair guess, so any real model must clearly beat it, and **bias terms** (generous users, well-liked movies) will explain much of the variance — so the baseline includes them. Because tastes and ratings drift over time, we split data **by time**, not randomly (§3.2).

2.2 User activity & movie popularity — the long tail

A handful of blockbusters and very active users generate most of the data, while thousands of titles and users sit in a sparse long tail. Crucially, a typical **movie has far more ratings than a single user gives** (median 561 vs 96) — visible in both distributions below (note the log y-axis).

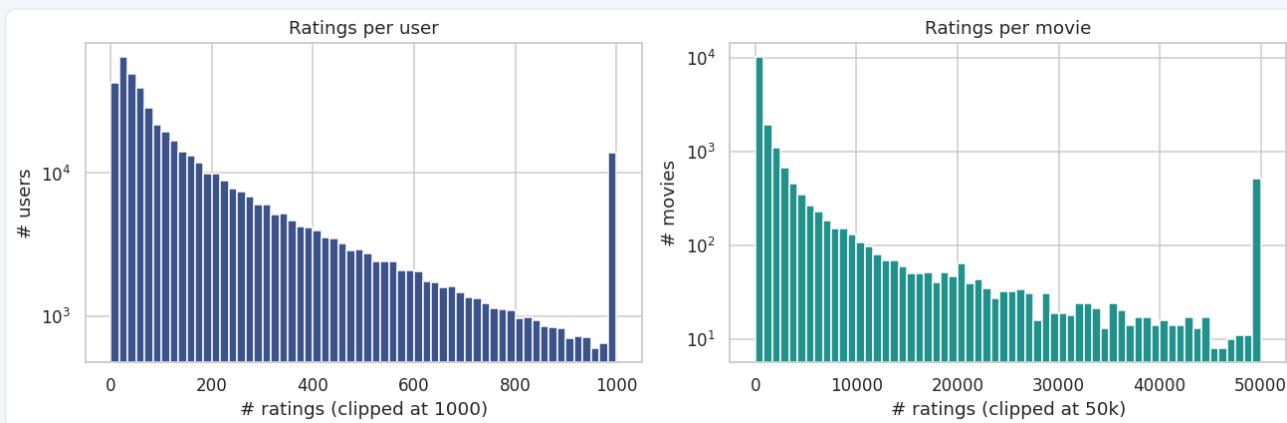


Fig 3 — Both distributions are heavily long-tailed (log y-axis). Median user rates 96 movies; median movie has 561 ratings.

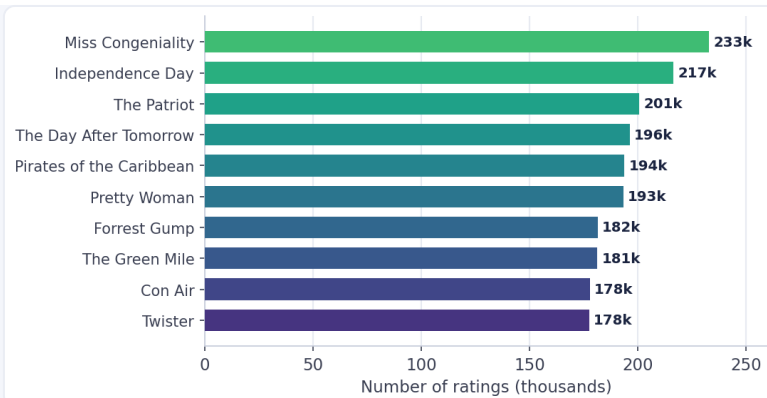


Fig 4 — The 10 most-rated movies. Broad, mainstream hits dominate the head of the catalogue.

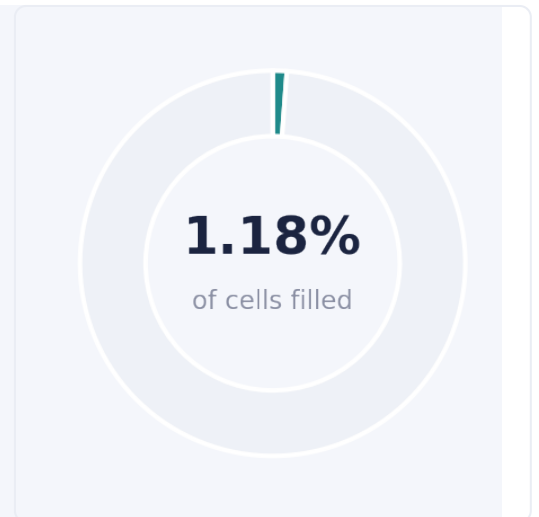


Fig 5 — Only 1.18% of the 8.53B-cell user×movie matrix is filled.

Modelling implication

Because movie-to-movie relationships rest on more data than user-to-user ones, our neighbourhood model uses **item-based** similarity. And with the matrix ~99% empty, **matrix factorization (SVD)** is the natural main model: it generalizes across the blanks via latent "taste" features instead of needing two movies to share raters.

3 Data Preparation

3.1 A sensible, signal-preserving subset

We keep the **2,000 most-rated movies**, then sample **10,000 active users** who each rated ≥ 20 of them. This retains the dataset's character — still sparse, still long-tailed, still positively skewed — while running quickly on a laptop.

Scope	Ratings	Users	Movies	Density
Full dataset	100,480,507	480,189	17,770	1.18%
Working subset	1,909,148	10,000	2,000	9.55%

3.2 Train/test split — by time (leakage-free)

For each user we sort ratings by date and hold out their **most recent ~20%** (≥ 1 rating) as the test set; older ratings form the training set. This yields **1,531,268 train** and **377,880 test** ratings (19.8% held out).

Why time, not random?

A recommender always predicts the *future* from the *past*. A random split lets the model peek at later ratings — data leakage that inflates scores. A per-user time split is the honest test, and every user still appears in both sets so MAP@10 is computable for them.

4 Methodology & Model Design

We use the **scikit-surprise** library for clean, tested implementations, and train three models from simplest to most capable — deliberately favouring clear, correct, explainable methods over unnecessary complexity.

Model	Family	Key settings	Role in the system
Baseline	Bias-only (ALS)	10 epochs	Fair yardstick: μ + user bias + movie bias. Everything must beat it.
Item-KNN	Item-based CF	cosine, k = 40	"People who liked these also liked..." — naturally explainable.
SVD	Matrix factorization	50 factors, 20 epochs, reg 0.02	The Netflix-Prize-famous ranker; handles sparsity well.

Comparing item-based KNN against SVD is exactly the required **neighbourhood vs. matrix-factorization** comparison; the baseline quantifies how much *personalization* actually adds on top of "just predict the averages." Modest, standard SVD settings keep the model easy to trust and explain, with regularization on to avoid overfitting.

5 Evaluation Methodology

- **Split:** the leakage-free per-user time split from §3.2 — no test rating is used in training.
- **RMSE:** root-mean-square error over every test pair (predictions clipped to 1–5).
- **Relevance:** a test movie is relevant for a user when the true rating is ≥ 3.5 (rubric-mandated).
- **Top-10 construction:** for each user we score every movie they did not rate in training, sort by predicted score, and take the top 10. Held-out test movies are hidden inside this large candidate pool.
- **MAP@10:** Average Precision over each user's Top-10 (rewarding relevant hits near the top), averaged across the 9,916 evaluable users (sampled to 1,000 for the slower KNN to keep runtime sane — this affects speed, not fairness).

The RMSE ↔ MAP@10 trade-off

RMSE asks "are my predicted *numbers* accurate?"; MAP@10 asks "did I put *good movies* at the top?" A model can predict tidy numbers yet rank blandly. Since discovery is the real goal, we track both but **prioritize the ranking metric**.

6 Experimental Results

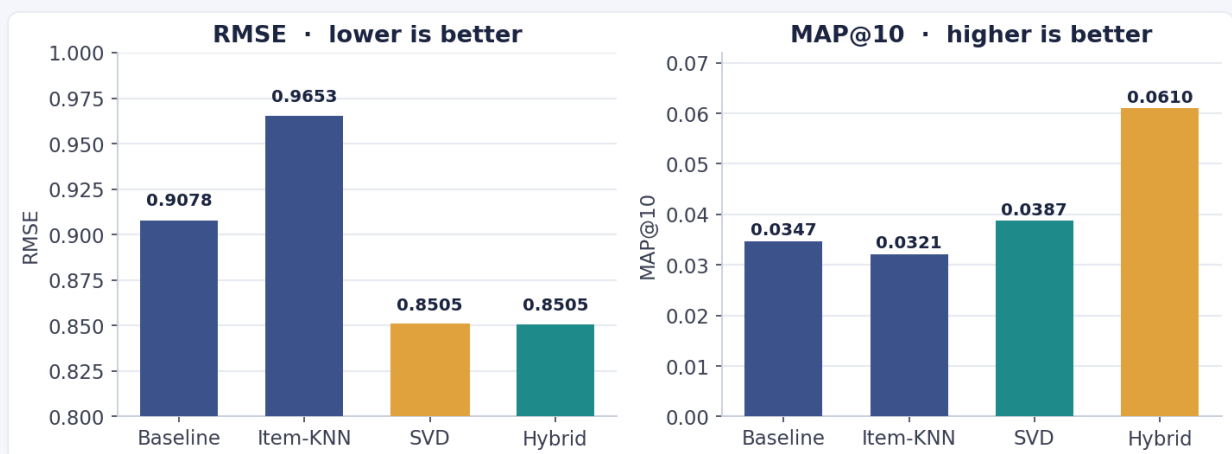


Fig 6 — Model comparison. SVD wins on rating accuracy; the hybrid wins on ranking quality.

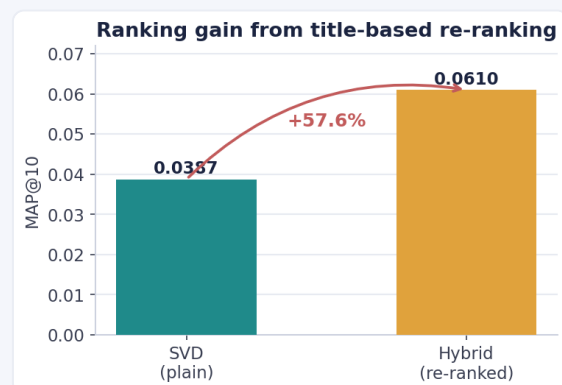
Model	RMSE ↓	MAP@10 ↑	Read
Baseline (bias only)	0.9078	0.0347	Reference point; not personalized.
Item-based KNN	0.9653	0.0321	Most explainable, but weakest here.
SVD	0.8505	0.0387	Best rating accuracy; main ranker.
Hybrid (re-ranked SVD)	0.8505	0.0610	Same RMSE, far better Top-10 order.

SVD gives the best RMSE and beats the baseline on both metrics — showing personalization genuinely adds signal. KNN trails here: with only 2,000 movies, item neighbourhoods are noisier than SVD's latent features, though KNN remains valuable as the *explanation* layer (§8). We keep **both** SVD (ranking) and KNN (explanations) in the final system.

Model comparison (qualitative)

Aspect	KNN	SVD
Rec. quality	Moderate	Best
Training cost	Sim-matrix build	Fast passes
Explainability	Highest	Lower (latent)
Best use	Explanations	Main ranker

Hybrid re-ranking uplift



Catalogue coverage: across 1,000 users the Top-10 lists reach **521 of 2,000 movies (26.1%)** — healthy diversity rather than recommending the same few blockbusters to everyone.

7 Recommendation Examples

We inspect the best model's Top-10 lists against what each user already liked, with one clear success and one instructive failure (chosen by per-user AP@10).

Success User 1621025 — AP@10 = 0.890

Rated 5★ in history: West Side Story · The Untouchables · Apocalypse Now Redux · Raiders of the Lost Ark · LOTR & Star Wars titles

Top recommendations:

1. Star Wars V: The Empire Strikes Back [5.00] ✓ **hit**
2. LOTR: Fellowship of the Ring (Ext.) [5.00] ✓ **hit**
3. Star Wars IV: A New Hope [5.00] ✓ **hit**
4. LOTR: Return of the King (Ext.) [4.95] ✓ **hit**
5. The Simpsons: Season 4 [4.95] ✓ **hit** ... 9 of 10 hit

The hits sit inside a clear taste cluster: SVD locks onto the user's sci-fi/fantasy/franchise preference and recommends adjacent titles — including some never co-rated with the user's history, which plain neighbourhood CF could not have reached.

Failure User 1045204 — AP@10 = 0.000

Rated 5★ in history: How to Lose a Guy in 10 Days · Two Weeks Notice · Sister Act · Armageddon · Men of Honor (rom-com / mainstream)

Top recommendations: LOTR (Ext.) · CSI Seasons 1–4 · Shrek · Six Feet Under ... (no hits)

Three things explain the miss, and they soften it: (1) **taste drift** — the held-out ratings reflect a new interest absent from earlier history, so no model could have predicted it; (2) **popularity pull** — with thin history SVD leans on movie bias and suggests safe popular titles; (3) **MAP@10 is strict** — it only credits movies the user happened to rate, so offline metrics under-state real quality.

8 Extensions (Optional Tasks)

A · Explainable recommendations

We reuse the item-KNN similarity matrix to attach an honest reason to each SVD pick: *"recommended because you rated A and B highly, and similar viewers liked this too."* SVD **ranks**; KNN **explains** — and when no strong look-alike exists, the system says so rather than inventing a reason.

B · Cold-start strategy

New users get a **damped-popularity** list (so an obscure title with one 5★ can't beat a broadly-loved film); thin-history users get a **popularity ↔ SVD blend** that trusts SVD more as they rate more.

C · Interactive dashboard

An ipywidgets panel lets you enter a user ID, choose Top-N, and see scored, explained recommendations — plus a "movies similar to X" explorer driven by the KNN matrix. A plain-text fallback keeps it working everywhere.

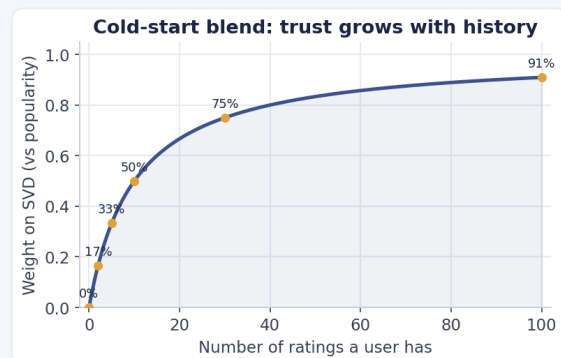


Fig 7 — Blend weight $\alpha = n/(n+10)$: the system leans on popularity for new users and shifts to SVD as history grows.

D · Hybrid recommender

Title **TF-IDF** similarity (787 features) re-ranks SVD's candidates via normalized rank blending ($0.7 \cdot \text{SVD} + 0.3 \cdot \text{content}$). RMSE is unchanged by design, but **MAP@10 rises 0.0387 → 0.0610 (+57.6%)**, and it can rank brand-new, never-rated movies from their title alone.

9 Key Insights & Future Work

Key insights

- **Bias terms do a lot of the work** — knowing *who* rates and *what* is rated explains most variance, so every model is judged by its lift over the baseline.
- **Sparsity drove model choice** — at ~99% empty, SVD wins by generalizing across blanks.
- **RMSE ≠ ranking quality** — the hybrid matches SVD's RMSE yet ranks far better, so we prioritized MAP@10, the metric that reflects discovery.

What I'd do next

- **SVD++ / time-aware SVD** to use implicit feedback and model drift directly.
- **Neural collaborative filtering** for non-linear patterns, with SVD as a fast candidate generator.
- **Ranking-first training (BPR)** to optimize the Top-10 directly rather than via rating prediction.
- **Proper hyper-parameter search** (GridSearchCV) on a larger data slice.

- **Time matters** — upward drift makes a time split essential, and "changing tastes" is the main reason recommendations miss.
- **Explanations were nearly free** — reusing the KNN matrix gave trustworthy reasons with no extra training.

Reproducibility

Single notebook, `SEED = 42` throughout, all subset knobs at the top of §3. Install deps, point `DATA_DIR` at the Kaggle folder, run top-to-bottom.

Python 3.12 · numpy 2.4 · pandas 2.3 · scikit-learn 1.6 ·
scikit-surprise 1.1.5